

Unit 04 - Introduction to Federated Learning

March 13, 2025

1 Federated Learning

Federated learning is a machine learning paradigm that enables decentralized training of a shared model by multiple clients while preserving data privacy. The main idea behind this new paradigm is that each client trains a local model on its own data and then sends only the model updates to a central server, rather than sending the raw data. This allows the model to be trained on a large amount of data without compromising data privacy.

Federated learning was first proposed by Google in 2016 (McMahan et al., 2016) and has since been applied in various fields, such as healthcare (Hard et al., 2018), finance (Yoon et al., 2018), and natural language processing (Li et al., 2020). For example, federated learning could be used to train a model that can make personalized recommendations for each user without requiring the raw data from each user to be shared with a central server. This is the mechanism by which the model is trained on data, while adhering to data privacy requirements.

Federated learning is a machine learning approach that offers numerous advantages over traditional centralized methods. Firstly, by leveraging distributed data stores, federated learning can scale to handle significantly larger datasets. Secondly, it prioritizes data privacy by avoiding the transmission of raw data to a central server. Finally, federated learning enables collaboration among multiple clients, allowing them to jointly train a shared model without compromising the security of their individual data. Overall, these benefits make federated learning a promising approach for machine learning in fields where data privacy is of the utmost importance.

Federated learning is a process in which a central server distributes a machine learning model to multiple devices. Each device trains the model on its local data and sends the updated model back to the central server. The central server then aggregates the updates from each device to improve the global model. This process is repeated until the model converges and can generate accurate predictions on new data. The key concepts within this process are:

- **Client:** refers to a device or edge node that holds a local dataset and actively participates in the training of the federated model.
- **Server:** represents the central entity that coordinates the training of the federated model and receives model updates from the clients to aggregate into a new version of the global model.
- **Federated dataset:** the collection of decentralized datasets from different clients that are used to train the federated model through collaborative learning.
- **Federated model:** a machine learning model that is trained on the federated dataset using federated learning to make accurate predictions on new data while preserving the privacy of each client's data.
- **Federated optimization:** refers to the process of training the federated model using the decentralized data and model updates from the clients, which enables the model to generalize

better on unseen data while preserving the privacy of the clients.

- **Aggregation:** the process of combining the model updates received from the clients into a new version of the global model. This can be done using various methods such as weighted averaging or other approaches.
- **Rounds:** refer to the number of times a federated model is distributed among clients after performing an aggregation to train the model further. The process is repeated until the model converges and achieves a satisfactory level of accuracy.

Federated learning is a relatively new approach, and as such, there are few libraries available that have adapted to it. The main actors in this space are TensorFlow Federated, PySyft, OpenMined, and Flower. Of these, TensorFlow Federated is a notable mention, although it is currently only a theoretical approach, as it does not allow for the deployment of the solution and only simulates the federated space. In contrast, Flower allows for the distribution of federated learning, although the necessary modifications can be somewhat challenging. For this tutorial, we have chosen Flower due to its more user-friendly approach and potential for future use.

2 Introduction to Flower (FLWR)

Flower is a Python library that offers tools for implementing the communication and coordination aspects of federated learning. Its design emphasizes ease of use and scalability. It's important to note that Flower is not a learning framework in itself, and as such, it wraps other machine learning frameworks like TensorFlow, PyTorch, or Scikit-learn in the communication layer to enable federated learning.

To use Flower for federated learning, you will need to install the library:

```
[1]: try:
      import flwr as fl
    except ImportError:
      !pip install flwr[simulation]
      import flwr as fl # Import again after installation

    try:
      import tensorflow as tf
    except ImportError:
      !pip install tensorflow
      import tensorflow as tf # Import again after installation
```

When setting up a simulation environment, it's crucial to use the `[simulation]` keyword with the `!pip install flwr[simulation]` command. This ensures that the necessary dependencies for simulating a federated learning environment, such as the `flwr_simulation` package, are also installed.

Important Note: In a distributed setup, use `!pip install flwr` on both the server and all client devices to ensure consistency.

Installation Guide:

- For the most up-to-date and comprehensive installation instructions, refer to the official Flower documentation: [Link to official Flower documentation](#)

After installing the `flwr` package, import it into your Python code as follows:

```
[2]: import flwr as fl
import tensorflow as tf
```

FLWR provides a range of classes and functions that you can use to set up a federated learning environment, train and evaluate a model, and implement regular updates to the model. For more information, refer to the FLWR [documentation](#). Before proceeding, it's important to note that the model you define must be serializable so that it can be sent through the network. Not all models are suitable for federated learning. For this example, we'll be using an Artificial Neural Network (ANN) based on TensorFlow, specifically Keras. These models are generally lightweight and well-suited for serialization compared to some other model types.

```
[3]: # Define a simple model using TensorFlow
def generate_ann():
    model = tf.keras.Sequential(
        [
            tf.keras.layers.Flatten(input_shape=(32, 32, 3)), #input data.
            ↪32x32 color images with 3 channels
            tf.keras.layers.Dense(64, activation="relu"), #hidden layer
            tf.keras.layers.Dense(64, activation="relu"), #hidden layer
            tf.keras.layers.Dense(10, activation="softmax"), #output layer. 10
            ↪classes
        ]
    )

    model.compile(
        loss=tf.keras.losses.sparse_categorical_crossentropy,
        optimizer=tf.keras.optimizers.Adam(),
        metrics=["accuracy"],
    )
    return model
```

As we will be using a Deep Learning model defined in TensorFlow, it's recommended to load the data into a `Dataset` class to enable the framework to leverage any available hardware acceleration (such as a GPU on the nodes). However, due to some limitations of the framework to serialize the data, it has to be done manually with the following lines of code

```
[4]: import numpy as np
import tensorflow as tf

NUM_CLIENTS = 5

def unison_shuffled_copies(a, b):
    assert len(a) == len(b)
    p = np.random.permutation(len(a))
    return a[p], b[p]
```

```

def split_index(a, n):
    s = np.array_split(np.arange(len(a)), n)
    return s

# Code to load the dataset
def load_datasets(num_clients: int):
    # Distribute it to train and test set
    (x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
    # Normalize data
    x_train = x_train.astype("float32") / 255.0
    x_test = x_test.astype("float32") / 255.0

    x_train, y_train = x_train[:10_000], y_train[:10_000]
    x_test, y_test = x_test[:1000], y_test[:1000]

    # Randomize the datasets
    x_train, y_train = unison_shuffled_copies(x_train, y_train)
    x_test, y_test = unison_shuffled_copies(x_test, y_test)

    # Split training set into NUM_CLIENTS partitions to simulate the individual
    ↪ dataset
    train_index = split_index(x_train, num_clients)
    test_index = split_index(x_test, num_clients)

    # Split each partition
    train_ds = []
    val_ds = []
    test_ds = []
    for cid in range(num_clients):
        val_size = len(train_index[cid]) // 10
        train_input_data, train_output_data = x_train[train_index[cid]], ↪
        ↪ y_train[train_index[cid]]
        val_input_data, val_output_data = train_input_data[:val_size], ↪
        ↪ train_output_data[:val_size]
        train_input_data, train_output_data = train_input_data[val_size:], ↪
        ↪ train_output_data[val_size:]
        train_dataset = (train_input_data, train_output_data)
        val_dataset = (val_input_data, val_output_data)
        test_dataset = (x_test[test_index[cid]], y_test[test_index[cid]])
        train_ds.append(train_dataset)
        val_ds.append(val_dataset)
        test_ds.append(test_dataset)
    return train_ds, val_ds, test_ds

trainloaders, valloaders, testloader = load_datasets(NUM_CLIENTS)

```

```
/Users/albertolandi/anaconda3/envs/ml2/lib/python3.12/site-  
packages/keras/src/utils/file_utils.py:115: DeprecationWarning: Python 3.14  
will, by default, filter extracted tar archives and reject files or modify their  
metadata. Use the filter argument to control this behavior.  
    archive.extractall()
```

Before proceeding, it's crucial to perform a basic test to validate the correctness of our approach. This involves training the model on the dataset of a single client. The purposes are: 1. **Checks Model Training Functionality**: It ensures that the model can be successfully trained on the data of an individual client. Identifies 2. **Potential Issues Early**: It helps to detect and address any errors or unexpected behavior in the data loading or model training processes before moving on to the more complex federated learning implementation

```
[5]: model = generate_ann()  
  
model.fit(trainloaders[0][0],trainloaders[0][1], epochs=1, batch_size=32,  
    ↪steps_per_epoch=3)  
loss, accuracy = model.evaluate(valloaders[0][0], valloaders[0][1])  
print(f"The resulting loss is {loss} for an accuracy in test of {accuracy*10:.  
    ↪4f}%")
```

```
/Users/albertolandi/anaconda3/envs/ml2/lib/python3.12/site-  
packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an  
`input_shape`/`input_dim` argument to a layer. When using Sequential models,  
prefer using an `Input(shape)` object as the first layer in the model instead.  
    super().__init__(**kwargs)
```

```
3/3          0s 3ms/step -  
accuracy: 0.1250 - loss: 2.4288  
7/7          0s 827us/step -  
accuracy: 0.1058 - loss: 2.4598  
The resulting loss is 2.409999132156372 for an accuracy in test of 1.1000%
```

Important: *The following example is thought to be executed in a terminal*

Starting from that point, that it works on a particular data, the federated process is going to split this process in several machines and for that it is going to require the definition of a couple of additional elements. Therefore, let's introduce the two pieces of the puzzle: the **Client** and the **Server**.

Flower starts a **Server** to coordinate the client devices and perform the orchestration of the model. The server interacts with clients through an interface called **Client**. When the server selects a particular client for training, it sends training instructions over the network. The client receives those instructions and calls one of the **Client** methods to run your code, which in this case involves training the neural network that we defined earlier.

Flower provides a convenient class called **NumPyClient**, which simplifies the implementation of the **Client** interface when your workload uses Keras. The **NumPyClient** interface defines three methods that can be implemented in the following way:

```
#Create a class to contain the details of the client and be the interface
```

```

class MyClient(fl.client.NumPyClient):
    def __init__(self, net, train_dataset, test_dataset):
        self.model = net
        self.trainloader = train_dataset
        self.valloader = test_dataset

    def get_parameters(self, config):
        return self.model.get_weights()

    def fit(self, parameters, config):
        self.model.set_weights(parameters)
        self.model.fit(self.trainloader[0], self.trainloader[1], epochs=1, batch_size=32, steps=
        return self.model.get_weights(), len(self.trainloader[0]), {})

    def evaluate(self, parameters, config):
        self.model.set_weights(parameters)
        loss, accuracy = self.model.evaluate(self.valloader[0], self.valloader[1])
        return loss, len(self.valloader[0]), {"accuracy": float(accuracy)}

```

In the preceding code, we defined the required functions for the client in this particular case. With these functions in place, we can now start a client using the following code:

```

# Start the client
model=generate_ann()

```

```

fl.client.start_client(server_address=f"localhost:8080", client=MyClient(model,trainloaders[0])

```

Important: In order to run the client you must need also a server running!!! You will executed both in separated terminals

The string `localhost:8080` specifies the server to which the client should connect. In this case, as the code is being run on the same machine as the server, this address is sufficient. In a truly federated workload, the only thing that needs to be changed is the `server_address` to point the client to the correct server.

Note that Jupyter usually runs on port 8080, **so you will need to use another available port if Jupyter server is running.**

The other essential piece of the puzzle is the class that will contain the server. This will be in a separate file, for example, `server.py`, and its contents should look something like this:

```

import flwr as fl

fl.server.start_server(config=fl.server.ServerConfig(num_rounds=3))

```

Important You can use another port for running the server if you fix the following parameter in the `start_server` function: `server_address="localhost:8080"`

In this particular case, we can run two clients and a server in separate terminals of the machine. Running two client instances is as simple as executing the `python client.py` command twice in separate terminals, while the server can be started with the `python server.py` command.

Upon starting the server, we should receive an output similar to:

```
INFO flwr 2023-03-01 14:58:16,353 | app.py:139 | Starting Flower server, config: ServerConfig(
INFO flwr 2023-03-01 14:58:16,362 | app.py:152 | Flower ECE: gRPC server running (3 rounds), S
INFO flwr 2023-03-01 14:58:16,362 | server.py:86 | Initializing global parameters
INFO flwr 2023-03-01 14:58:16,362 | server.py:270 | Requesting initial parameters from one ran
INFO flwr 2023-03-01 14:58:24,152 | server.py:274 | Received initial parameters from one random
INFO flwr 2023-03-01 14:58:24,153 | server.py:88 | Evaluating initial parameters
INFO flwr 2023-03-01 14:58:24,153 | server.py:101 | FL starting
DEBUG flwr 2023-03-01 14:58:26,118 | server.py:215 | fit_round 1: strategy sampled 2 clients (
DEBUG flwr 2023-03-01 14:58:27,041 | server.py:229 | fit_round 1 received 2 results and 0 fail
WARNING flwr 2023-03-01 14:58:27,076 | fedavg.py:242 | No fit_metrics_aggregation_fn provided
DEBUG flwr 2023-03-01 14:58:27,076 | server.py:165 | evaluate_round 1: strategy sampled 2 clie
DEBUG flwr 2023-03-01 14:58:27,565 | server.py:179 | evaluate_round 1 received 2 results and 0
WARNING flwr 2023-03-01 14:58:27,565 | fedavg.py:273 | No evaluate_metrics_aggregation_fn prov
DEBUG flwr 2023-03-01 14:58:27,566 | server.py:215 | fit_round 2: strategy sampled 2 clients (
DEBUG flwr 2023-03-01 14:58:28,015 | server.py:229 | fit_round 2 received 2 results and 0 fail
DEBUG flwr 2023-03-01 14:58:28,027 | server.py:165 | evaluate_round 2: strategy sampled 2 clie
DEBUG flwr 2023-03-01 14:58:28,364 | server.py:179 | evaluate_round 2 received 2 results and 0
DEBUG flwr 2023-03-01 14:58:28,364 | server.py:215 | fit_round 3: strategy sampled 2 clients (
DEBUG flwr 2023-03-01 14:58:28,755 | server.py:229 | fit_round 3 received 2 results and 0 fail
DEBUG flwr 2023-03-01 14:58:28,769 | server.py:165 | evaluate_round 3: strategy sampled 2 clie
DEBUG flwr 2023-03-01 14:58:29,184 | server.py:179 | evaluate_round 3 received 2 results and 0
INFO flwr 2023-03-01 14:58:29,185 | server.py:144 | FL finished in 5.031599427999936
INFO flwr 2023-03-01 14:58:29,185 | app.py:202 | app_fit: losses_distributed [(1, 2.3956351280
INFO flwr 2023-03-01 14:58:29,185 | app.py:203 | app_fit: metrics_distributed {}
INFO flwr 2023-03-01 14:58:29,186 | app.py:204 | app_fit: losses_centralized []
```

With that, the first federated learning approach is completed. As you can see, the system goes through three rounds of fitting and evaluating on all clients before the results are retrieved, aggregated, and redistributed to the server.

2.0.1 Exercise

Implement the client and server code in separate files (**All the required files must be submitted together with the Notebook**). Next, execute a server and two clients from terminals. Finally, compare the results with those presented here. Were your results similar?

Note: The answer is expected to contain the corresponding terminal output resulting from the execution of your code.

Answer here:

Files can be found in the U04 Simulation folder.

Server terminal output:

```
(ml2) albertolandi@Albertos-MacBook-Air U04 Simulation % python server.py
WARNING : DEPRECATED FEATURE: flwr.server.start_server() is deprecated.
        Instead, use the `flower-superlink` CLI command to start a SuperLink as shown below:
```

```
$ flower-superlink --insecure
```

To view usage and all available options, run:

```
$ flower-superlink --help
```

Using `start\_server()` is deprecated.

This is a deprecated feature. It will be removed entirely in future versions of Flower.

```
INFO : Starting Flower server, config: num_rounds=3, no round_timeout
INFO : Flower ECE: gRPC server running (3 rounds), SSL is disabled
INFO : [INIT]
INFO : Requesting initial parameters from one random client
INFO : Received initial parameters from one random client
INFO : Starting evaluation of initial global parameters
INFO : Evaluation returned no results (`None`)
INFO :
INFO : [ROUND 1]
INFO : configure_fit: strategy sampled 2 clients (out of 2)
INFO : aggregate_fit: received 2 results and 0 failures
WARNING : No fit_metrics_aggregation_fn provided
INFO : configure_evaluate: strategy sampled 2 clients (out of 2)
INFO : aggregate_evaluate: received 2 results and 0 failures
WARNING : No evaluate_metrics_aggregation_fn provided
INFO :
INFO : [ROUND 2]
INFO : configure_fit: strategy sampled 2 clients (out of 2)
INFO : aggregate_fit: received 2 results and 0 failures
INFO : configure_evaluate: strategy sampled 2 clients (out of 2)
INFO : aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO : [ROUND 3]
INFO : configure_fit: strategy sampled 2 clients (out of 2)
INFO : aggregate_fit: received 2 results and 0 failures
INFO : configure_evaluate: strategy sampled 2 clients (out of 2)
INFO : aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO : [SUMMARY]
INFO : Run finished 3 round(s) in 135.70s
INFO : History (loss, distributed):
INFO : round 1: 2.3193126916885376
INFO : round 2: 2.3956093788146973
INFO : round 3: 2.30367910861969
INFO :
(ml2) albertolandi@Albertos-MacBook-Air U04 Simulation %
```

- When comparing both:

The results obtained were similar to those presented in the notebook, although the exact loss values differed slightly. This is expected due to the inherent randomness in model initialization and data partitioning in federated learning.

A comparison of the distributed losses is presented below:

Round	Obtained Loss	Example's Loss
1	2.3193	2.3956
2	2.3956	2.4264
3	2.3036	2.3015

While the numerical values are not identical, the general trend is consistent: the loss fluctuates and generally decreases across rounds, which indicates that the federated model is learning. The slight variations are likely attributable to differences in the random shuffling of the data and the initial weights of the model.

3 Updating parameters

The key element in this kind of approach is that the server sends the global model parameters to the client, and the client updates the local model with the parameters received from the server. It then trains the model on the local data, which changes the model parameters locally. After training, the updated model parameters are sent back to the server, or alternatively, only the gradients are sent back to the server, not the full model parameters.

In **flwr**, this communication is essentially done by two helper functions for loading and retrieving local parameters: **set_parameters** and **get_parameters**. This requirement fits well with non-state approaches such as **PyTorch** or **JAX**. As demonstrated in the previous example, **flwr** can also be used with **TensorFlow** or even **scikit-learn**.

As a result, the basic structure for any client using this library has the same format:

```
[6]: from typing import List

# Utility functions for the most common operations
def get_parameters(net) -> List[np.array]:
    return net.get_weights()

def set_parameters(net, parameters: List[np.ndarray]):
    net.set_weights(parameters)
    return net

def train(net, trainloader, epochs: int):
    net.fit(trainloader[0], trainloader[1],
            epochs=epochs, batch_size=32, steps_per_epoch=3)
```

```

    return net

def test(net, testloader):
    loss, accuracy = net.evaluate(testloader[0], testloader[1])
    return loss, accuracy

# Class to contain a Client
class FlowerClient(fl.client.NumPyClient):
    def __init__(self, cid, net, trainloader, valloader):
        self.cid = cid
        self.net = net
        self.trainloader = trainloader
        self.valloader = valloader

    def get_parameters(self, config):
        print(f"[Client {self.cid}] get_parameters")
        return get_parameters(self.net)

    def fit(self, parameters, config):
        print(f"[Client {self.cid}] fit, config: {config}")
        self.net = set_parameters(self.net, parameters)
        self.net = train(self.net, self.trainloader, epochs=1)
        return get_parameters(self.net), len(self.trainloader), {}

    def evaluate(self, parameters, config):
        print(f"[Client {self.cid}] evaluate, config: {config}")
        self.net = set_parameters(self.net, parameters)
        loss, accuracy = test(self.net, self.valloader)
        print(f"[Client {self.cid}] loss:{loss}, Client {self.cid} accuracy:
↪{accuracy}")
        return float(loss), len(self.valloader), {"accuracy": float(accuracy)}

```

In Flower, clients can be created by extending either the `flwr.client.Client` or `flwr.client.NumPyClient` classes. In the previous example, we used `NumPyClient` because it is easier to implement and requires less code as a template. Along with the extended class, there are three main methods that need to be implemented:

- **get_parameters**: Returns the current local model parameters.
- **fit**: Receives model parameters from the server, trains the model parameters on the local data, and returns the (updated) model parameters to the server.
- **evaluate**: Receives model parameters from the server, evaluates the model parameters on the local data, and returns the evaluation result to the server.

As you can see, the `MyClient` class implemented in the previous example follows this same structure and the difference is the *id* of the client which is stored for later convenience use in accessing the data.

Be aware: Sometimes, especially when we are simulating multiple clients on a single device, it can be useful to use a function to create the client when it is required. This is particularly important in stateless frameworks, such as PyTorch, which can benefit from a more efficient implementation that creates clients only when they are required for training or evaluation. For example, the following code loads different examples for each client before discarding them:

```
[7]: from flwr.common import Context
      from flwr.client import ClientApp

      def client_fn(context: Context):
          """Returns a FlowerClient containing its data partition."""

          # Create the model
          net = generate_ann()
          #get the identification
          partition_id = int(context.node_config["partition-id"])
          #Take the appropriate part of the dataset
          trainloader = trainloaders[int(partition_id)]
          valloader = valloaders[int(partition_id)]
          #Create and return the Client
          return FlowerClient(partition_id, net, trainloader, valloader).to_client()
```

From version 1.13 onward, in order to run the simulation it is mandatory that the clients are contained inside a `ClientApp`. This represents each one of the nodes that are going to be used and called later by the simulation function.

```
[8]: # Construct the ClientApp passing the client generation function
      client_app = ClientApp(client_fn=client_fn)
```

Note that `myClient` cannot be used in the same sense because of the state that it keeps internally through the function `generate_ann`. However, if this state is removed, it can be used in the same way similarly to this `ClientFlower`.

The clients are now set up to load, fit, and evaluate. However, we need to integrate the results from the different clients. In Flower terminology, this is known as a strategy, such as the *Federated Average (FedAvg)* strategy. In a first approach, we can use the built-in implementations of the framework, although custom strategies can also be used. Additionally, all those elements have to be set in an object in the Flower ecosystem with a `ServerApp` in order to run them. It has a similar signature to `client_fn` but, instead of returning a client object, it returns all the components needed to run the server-side logic in Flower. Let's see an example:

```
[9]: from flwr.server import ServerApp, ServerAppComponents

      num_rounds = 3

      def server_fn(context: Context):
          # instantiate the model
          model = generate_ann()
```

```

params = get_parameters(model) # The federated model initial parameters
del model # this is not require but it saves a good amount of memory
# Convert model parameters to flwr.common.Parameters
global_model_init = fl.common.ndarrays_to_parameters(params)

# Create FedAvg strategy
strategy = fl.server.strategy.FedAvg(
    fraction_fit=1.0, # Sample 100% of available clients for training
    fraction_evaluate=0.5, # Sample 50% of available clients for
    ↪evaluation
    min_fit_clients=NUM_CLIENTS, # Never sample less than NUM_CLIENTS
    ↪clients for training
    min_evaluate_clients=NUM_CLIENTS//2, # Never sample less than
    ↪NUM_CLIENTS//2 clients for evaluation
    min_available_clients=NUM_CLIENTS, # Wait until all NUM_CLIENTS
    ↪clients are available
    initial_parameters=fl.common.ndarrays_to_parameters(params), #
    ↪Initial parameters
)

# Define ServerConfig
config=fl.server.ServerConfig(num_rounds=5)

# Return the configuration and strategy for this server
return ServerAppComponents(strategy=strategy, config=config)

# Create Server
server_app = ServerApp(server_fn=server_fn)

```

Now we have the two pieces of the puzzle. Therefore, the simulation engine can be called.

```

[10]: fl.simulation.run_simulation(
    server_app=server_app, client_app=client_app, num_supernodes=NUM_CLIENTS
)

```

```

INFO :      Starting Flower ServerApp, config: num_rounds=5, no
round_timeout
INFO :
INFO :      [INIT]
INFO :      Using initial global parameters provided by strategy
INFO :      Starting evaluation of initial global parameters
INFO :      Evaluation returned no results (`None`)
INFO :
INFO :      [ROUND 1]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)
(ClientAppActor pid=60501)

```

```

/Users/albertolandi/anaconda3/envs/ml2/lib/python3.12/site-
packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
(ClientAppActor pid=60501) super().__init__(**kwargs)

(ClientAppActor pid=60500) [Client 3] fit, config: {}
3/3          1s 8ms/step -
accuracy: 0.0846 - loss: 2.7059

INFO :      aggregate_fit: received 5 results and 0 failures
WARNING : No fit_metrics_aggregation_fn provided
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)

(ClientAppActor pid=60501) [Client 2] evaluate, config: {}
(ClientAppActor pid=60501) [Client 2] loss:2.4528427124023438, Client 2
accuracy:0.13500000536441803

INFO :      aggregate_evaluate: received 2 results and 0 failures
WARNING : No evaluate_metrics_aggregation_fn provided
INFO :
INFO :      [ROUND 2]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)

7/7          0s 568us/step -
accuracy: 0.1097 - loss: 2.4136

INFO :      aggregate_fit: received 5 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 3]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)

(ClientAppActor pid=60501) [Client 4] fit, config: {} [repeated 9x
across cluster] (Ray deduplicates logs by default. Set RAY_DEDUP_LOGS=0 to
disable log deduplication, or see https://docs.ray.io/en/master/ray-observability/user-guides/configure-logging.html#log-deduplication for more
options.)

7/7          0s 1ms/step -
accuracy: 0.0800 - loss: 2.2840 [repeated 12x across cluster]
(ClientAppActor pid=60499) [Client 4] evaluate, config: {}

[repeated 3x across cluster]

1/3          0s 424ms/step -
accuracy: 0.0938 - loss: 2.2489
(ClientAppActor pid=60499) [Client 4] loss:2.283036708831787, Client 4
accuracy:0.08500000089406967 [repeated 3x across cluster]

3/3          0s 3ms/step -

```

accuracy: 0.1185 - loss: 2.2964

```
(ClientAppActor pid=60501) WARNING:tensorflow:5 out of the last 13
calls to <function
TensorFlowTrainer.make_train_function.<locals>.one_step_on_iterator at
0x17f8bfb00> triggered tf.function retracing. Tracing is expensive and the
excessive number of tracings could be due to (1) creating @tf.function
repeatedly in a loop, (2) passing tensors with different shapes, (3) passing
Python objects instead of tensors. For (1), please define your @tf.function
outside of the loop. For (2), @tf.function has reduce_retracing=True option that
can avoid unnecessary retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling\_retracing and
https://www.tensorflow.org/api\_docs/python/tf/function for more details.
```

```
(ClientAppActor pid=60502)
/Users/albertolandi/anaconda3/envs/ml2/lib/python3.12/site-
packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model
instead. [repeated 3x across cluster]
```

```
(ClientAppActor pid=60502) super().__init__(**kwargs) [repeated
```

```
3x across cluster]
```

```
INFO :      aggregate_fit: received 5 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 4]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)
```

```
(ClientAppActor pid=60502) [Client 1] fit, config: {} [repeated 9x
across cluster]
```

```
(ClientAppActor pid=60501) WARNING:tensorflow:5 out of the last 13
calls to <function
TensorFlowTrainer.make_train_function.<locals>.one_step_on_iterator at
0x39605f600> triggered tf.function retracing. Tracing is expensive and the
excessive number of tracings could be due to (1) creating @tf.function
repeatedly in a loop, (2) passing tensors with different shapes, (3) passing
Python objects instead of tensors. For (1), please define your @tf.function
outside of the loop. For (2), @tf.function has reduce_retracing=True option that
can avoid unnecessary retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling\_retracing and
https://www.tensorflow.org/api\_docs/python/tf/function for more details.
```

```
INFO :      aggregate_fit: received 5 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 5]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)
```

```

7/7          0s 536us/step -
accuracy: 0.1791 - loss: 2.2584
3/3          0s 2ms/step -
accuracy: 0.1576 - loss: 2.2680    [repeated 11x across cluster]
(ClientAppActor pid=60500) [Client 1] evaluate, config: {}

[repeated 4x across cluster]
(ClientAppActor pid=60500) [Client 1] loss:2.244774103164673, Client 1
accuracy:0.17000000178813934 [repeated 4x across cluster]

(ClientAppActor pid=60500) WARNING:tensorflow:5 out of the last 13
calls to <function
TensorFlowTrainer.make_train_function.<locals>.one_step_on_iterator at
0x1770b09a0> triggered tf.function retracing. Tracing is expensive and the
excessive number of tracings could be due to (1) creating @tf.function
repeatedly in a loop, (2) passing tensors with different shapes, (3) passing
Python objects instead of tensors. For (1), please define your @tf.function
outside of the loop. For (2), @tf.function has reduce_retracing=True option that
can avoid unnecessary retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling\_retracing and
https://www.tensorflow.org/api\_docs/python/tf/function for more details.

[repeated 2x across cluster]

(ClientAppActor pid=60500) [Client 4] fit, config: {} [repeated 6x
across cluster]

INFO :      aggregate_fit: received 5 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [SUMMARY]
INFO :      Run finished 5 round(s) in 28.14s
INFO :      History (loss, distributed):
INFO :          round 1: 2.425778865814209
INFO :          round 2: 2.283255696296692
INFO :          round 3: 2.2986581325531006
INFO :          round 4: 2.2372995615005493
INFO :          round 5: 2.250061869621277
INFO :

7/7          0s 604us/step -
accuracy: 0.1451 - loss: 2.2488

(ClientAppActor pid=60501) WARNING :   Manually terminating
ClientAppActor
(ClientAppActor pid=60500) WARNING:tensorflow:5 out of the last 13
calls to <function
TensorFlowTrainer.make_train_function.<locals>.one_step_on_iterator at
0x17726bce0> triggered tf.function retracing. Tracing is expensive and the

```

excessive number of tracings could be due to (1) creating `@tf.function` repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your `@tf.function` outside of the loop. For (2), `@tf.function` has `reduce_retracing=True` option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

```
[repeated 3x across cluster]
(ClientAppActor pid=60502) WARNING : Manually terminating
ClientAppActor [repeated 3x across cluster]

7/7          0s 572us/step -
accuracy: 0.2247 - loss: 2.2556 [repeated 7x across cluster]
(ClientAppActor pid=60501) [Client 4] evaluate, config: {}

[repeated 2x across cluster]
(ClientAppActor pid=60501) [Client 4] loss:2.264850378036499, Client 4
accuracy:0.20000000298023224 [repeated 2x across cluster]
```

This code corresponds to the script running on the server, and it uses the simulation function to test this approach on a single device with the previously mentioned optimization to avoid overloading the device. The code generates `NUM_CLIENTS` clients and randomly selects all of them (`fraction_fit = 1.0`) to train the model on all of them. After receiving the updates from the clients, the server performs the aggregation strategy before returning the global model to the clients for the next 3 rounds.

Note: *Be aware that the simulation is a really resource consuming task. You can get some errors and warnings linked to that. There are different [configurations](#) that you can apply to configure the available resources for simulating the process.*

One point to highlight is that the framework is not only going to manage the `losses_distributed`, but none of the other metrics. Due to the diverse treatment of those measures, the framework cannot accurately handle the aggregation of these metrics. Users need to tell the framework how to handle and aggregate these custom metrics.

The strategy will then call these functions whenever it receives fit or evaluates metrics from clients. The two possible functions are `fit_metrics_aggregation_fn` and `evaluate_metrics_aggregation_fn`. For example, the following code creates the weighted average, and the previous example can be adapted as follows:

```
[ ]: from typing import Tuple, List
      from flwr.common import Metrics

def weighted_average(metrics: List[Tuple[int, Metrics]]) -> Metrics:
    # Multiply accuracy of each client by number of examples used
    accuracies = [num_examples * m["accuracy"] for num_examples, m in metrics]
    examples = [num_examples for num_examples, _ in metrics]
    # Aggregate and return custom metric (weighted average)
    return {"accuracy": sum(accuracies) / sum(examples)}
```

```

def server_fn(context: Context):
    # instantiate the model
    model = generate_ann()
    params = get_parameters(model) # The federated model initial parameters
    del model # this is not require but it saves a good amount of memory
    # Convert model parameters to flwr.common.Parameters
    global_model_init = fl.common.ndarrays_to_parameters(params)

    # Create FedAvg strategy
    strategy = fl.server.strategy.FedAvg(
        fraction_fit=1.0, # Sample 100% of available clients for training
        fraction_evaluate=0.5, # Sample 50% of available clients for
        ↪evaluation
        min_fit_clients=NUM_CLIENTS, # Never sample less than NUM_CLIENTS
        ↪clients for training
        min_evaluate_clients=NUM_CLIENTS//2, # Never sample less than
        ↪NUM_CLIENTS//2 clients for evaluation
        min_available_clients=NUM_CLIENTS, # Wait until all NUM_CLIENTS
        ↪clients are available
        initial_parameters=fl.common.ndarrays_to_parameters(params), #
        ↪Initial parameters
        evaluate_metrics_aggregation_fn=weighted_average, # put the metric
        ↪aggregation for the evaluation
    )

    # Define ServerConfig
    config=fl.server.ServerConfig(num_rounds=5)

    # Return the configuration and strategy for this server
    return ServerAppComponents(strategy=strategy, config=config)

# Create Server
server_app = ServerApp(server_fn=server_fn)

fl.simulation.run_simulation(
    server_app=server_app, client_app=client_app, num_supernodes=NUM_CLIENTS
)

```

/Users/albertolandi/anaconda3/envs/ml2/lib/python3.12/site-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```

super().__init__(**kwargs)
INFO : Starting Flower ServerApp, config: num_rounds=5, no
round_timeout
INFO :

```

```

INFO :      [INIT]
INFO :      Using initial global parameters provided by strategy
INFO :      Starting evaluation of initial global parameters
INFO :      Evaluation returned no results (`None`)
INFO :
INFO :      [ROUND 1]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)
(ClientAppActor pid=60676)
/Users/albertolandi/anaconda3/envs/ml2/lib/python3.12/site-
packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
(ClientAppActor pid=60676) super().__init__(**kwargs)
(ClientAppActor pid=60677) [Client 3] fit, config: {}
3/3          0s 4ms/step -
accuracy: 0.1029 - loss: 2.5617

INFO :      aggregate_fit: received 5 results and 0 failures
WARNING : No fit_metrics_aggregation_fn provided
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)

(ClientAppActor pid=60677) [Client 1] evaluate, config: {}
(ClientAppActor pid=60677) [Client 1] loss:2.3321657180786133, Client 1
accuracy:0.125

INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 2]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)
INFO :      aggregate_fit: received 5 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)

(ClientAppActor pid=60677) [Client 4] fit, config: {} [repeated 9x
across cluster]

INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 3]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)

7/7          0s 574us/step -
accuracy: 0.0920 - loss: 2.3105 [repeated 13x across cluster]
1/3          0s 450ms/step -
accuracy: 0.0938 - loss: 2.3204
(ClientAppActor pid=60677) [Client 2] evaluate, config: {}

[repeated 3x across cluster]
(ClientAppActor pid=60676) [Client 3] loss:2.3395631313323975, Client 3
accuracy:0.08500000089406967 [repeated 3x across cluster]
3/3          0s 2ms/step -

```

accuracy: 0.1224 - loss: 2.3107

(ClientAppActor pid=60677) WARNING:tensorflow:5 out of the last 13 calls to <function TensorFlowTrainer.make_train_function.<locals>.one_step_on_iterator at 0x39036e200> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

(ClientAppActor pid=60678) /Users/albertolandi/anaconda3/envs/ml2/lib/python3.12/site-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead. [repeated 3x across cluster]

(ClientAppActor pid=60678) super().__init__(**kwargs) [repeated

3x across cluster]

INFO : aggregate_fit: received 5 results and 0 failures
INFO : configure_evaluate: strategy sampled 2 clients (out of 5)
INFO : aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO : [ROUND 4]
INFO : configure_fit: strategy sampled 5 clients (out of 5)

1/3 0s 424ms/step -
accuracy: 0.2500 - loss: 2.3052

(ClientAppActor pid=60678) [Client 0] fit, config: {} [repeated 9x

across cluster]

3/3 0s 2ms/step -
accuracy: 0.1771 - loss: 2.2963

(ClientAppActor pid=60677) WARNING:tensorflow:5 out of the last 13 calls to <function TensorFlowTrainer.make_train_function.<locals>.one_step_on_iterator at 0x17a721580> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

INFO : aggregate_fit: received 5 results and 0 failures
INFO : configure_evaluate: strategy sampled 2 clients (out of 5)

```

INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 5]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)

7/7          0s 625us/step -
accuracy: 0.1523 - loss: 2.2593 [repeated 12x across cluster]
(ClientAppActor pid=60679) [Client 3] evaluate, config: {}

[repeated 4x across cluster]
(ClientAppActor pid=60679) [Client 3] loss:2.267077684402466, Client 3
accuracy:0.16500000655651093 [repeated 4x across cluster]

(ClientAppActor pid=60678) WARNING:tensorflow:5 out of the last 13
calls to <function
TensorFlowTrainer.make_train_function.<locals>.one_step_on_iterator at
0x178224860> triggered tf.function retracing. Tracing is expensive and the
excessive number of tracings could be due to (1) creating @tf.function
repeatedly in a loop, (2) passing tensors with different shapes, (3) passing
Python objects instead of tensors. For (1), please define your @tf.function
outside of the loop. For (2), @tf.function has reduce_retracing=True option that
can avoid unnecessary retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling\_retracing and
https://www.tensorflow.org/api\_docs/python/tf/function for more details.

[repeated 3x across cluster]

(ClientAppActor pid=60679) [Client 1] fit, config: {} [repeated 5x
across cluster]

INFO :      aggregate_fit: received 5 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [SUMMARY]
INFO :      Run finished 5 round(s) in 24.41s
INFO :      History (loss, distributed):
INFO :          round 1: 2.3492780923843384
INFO :          round 2: 2.34087336063385
INFO :          round 3: 2.3386341333389282
INFO :          round 4: 2.2789441347122192
INFO :          round 5: 2.3168126344680786
INFO :      History (metrics, distributed, evaluate):
INFO :      {'accuracy': [(1, 0.12000000104308128),
INFO :          (2, 0.07249999977648258),
INFO :          (3, 0.13249999657273293),
INFO :          (4, 0.14000000432133675),
INFO :          (5, 0.08999999985098839)]}
INFO :

```

```

7/7          0s 742us/step -
accuracy: 0.0856 - loss: 2.2920 [repeated 7x across cluster]
(ClientAppActor pid=60679) [Client 3] evaluate, config: {}

[repeated 2x across cluster]
(ClientAppActor pid=60679) [Client 3] loss:2.316067695617676, Client 3
accuracy:0.08500000089406967 [repeated 2x across cluster]
(ClientAppActor pid=60677) [Client 4] fit, config: {}

(ClientAppActor pid=60679) WARNING:tensorflow:5 out of the last 13
calls to <function
TensorFlowTrainer.make_train_function.<locals>.one_step_on_iterator at
0x3861a7c40> triggered tf.function retracing. Tracing is expensive and the
excessive number of tracings could be due to (1) creating @tf.function
repeatedly in a loop, (2) passing tensors with different shapes, (3) passing
Python objects instead of tensors. For (1), please define your @tf.function
outside of the loop. For (2), @tf.function has reduce_retracing=True option that
can avoid unnecessary retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling\_retracing and
https://www.tensorflow.org/api\_docs/python/tf/function for more details.

[repeated 2x across cluster]

```

We will revisit the definition of custom strategies in the following unit to define our own strategy and attempt to minimize some of the challenges that federated learning must address.

3.0.1 Exercise

Now is your turn, why not you try to run your own architecture with this approach. Beaware of the high requirements when we are in a simulated environment.

Note: The objective of this exercise is to bring together the various topics covered by this notebook. This cell should contain all the necessary code for autonomous execution using Flower simulation.

```

[15]: import tensorflow as tf

# Code to load the dataset
import numpy as np

NUM_CLIENTS = 5

def unison_shuffled_copies(a, b):
    assert len(a) == len(b)
    p = np.random.permutation(len(a))
    return a[p], b[p]

def split_index(a, n):
    s = np.array_split(np.arange(len(a)), n)
    return s

```

```

def generate_ann():
    model = tf.keras.Sequential(
        [
            tf.keras.layers.Input(shape=(32, 32, 3)),
            tf.keras.layers.Conv2D(32, (3, 3), activation='relu',
↳padding='same'),
            tf.keras.layers.BatchNormalization(),
            tf.keras.layers.Conv2D(32, (3, 3), activation='relu',
↳padding='same'),
            tf.keras.layers.BatchNormalization(),
            tf.keras.layers.MaxPooling2D((2, 2)),
            tf.keras.layers.Dropout(0.25),

            tf.keras.layers.Conv2D(64, (3, 3), activation='relu',
↳padding='same'),
            tf.keras.layers.BatchNormalization(),
            tf.keras.layers.Conv2D(64, (3, 3), activation='relu',
↳padding='same'),
            tf.keras.layers.BatchNormalization(),
            tf.keras.layers.MaxPooling2D((2, 2)),
            tf.keras.layers.Dropout(0.25),

            tf.keras.layers.Flatten(),
            tf.keras.layers.Dense(128, activation='relu'),
            tf.keras.layers.BatchNormalization(),
            tf.keras.layers.Dropout(0.5),
            tf.keras.layers.Dense(10, activation='softmax'),
        ]
    )
    model.compile(
        loss=tf.keras.losses.sparse_categorical_crossentropy,
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
        metrics=["accuracy"],
    )
    return model

# Code to load the dataset
def load_datasets(num_clients: int):
    # Distribute it to train and test set
    (x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
    # Normalize data
    x_train = x_train.astype("float32") / 255.0
    x_test = x_test.astype("float32") / 255.0

    x_train, y_train = x_train[:10_000], y_train[:10_000]

```

```

x_test, y_test = x_test[:1000], y_test[:1000]

# Randomize the datasets
x_train, y_train = unison_shuffled_copies(x_train, y_train)
x_test, y_test = unison_shuffled_copies(x_test, y_test)

# Split training set into num_clients partitions to simulate the individual
↳dataset
train_index = split_index(x_train, num_clients)
test_index = split_index(x_test, num_clients)

# Split each partition
train_ds = []
val_ds = []
test_ds = []
for cid in range(num_clients):
    val_size = len(train_index[cid]) // 10
    train_input_data, train_output_data = x_train[train_index[cid]],
↳y_train[train_index[cid]]
    val_input_data, val_output_data = train_input_data[:val_size],
↳train_output_data[:val_size]
    train_input_data, train_output_data = train_input_data[val_size:],
↳train_output_data[val_size:]
    train_dataset = (train_input_data, train_output_data)
    val_dataset = (val_input_data, val_output_data)
    test_dataset = (x_test[test_index[cid]], y_test[test_index[cid]])
    train_ds.append(train_dataset)
    val_ds.append(val_dataset)
    test_ds.append(test_dataset)
return train_ds, val_ds, test_ds

trainloaders, valloaders, testloader = load_datasets(NUM_CLIENTS)

#TODO Client, client_fn, Server and simulation

class FlowerClient(fl.client.NumPyClient):
    def __init__(self, cid, net, trainloader, valloader):
        self.cid = cid
        self.net = net
        self.trainloader = trainloader
        self.valloader = valloader

    def get_parameters(self, config):
        print(f"[Client {self.cid}] get_parameters")
        return get_parameters(self.net)

```

```

def fit(self, parameters, config):
    print(f"[Client {self.cid}] fit, config: {config}")
    self.net = set_parameters(self.net, parameters)
    self.net = train(self.net, self.trainloader, epochs=5)
    return get_parameters(self.net), len(self.trainloader), {}

def evaluate(self, parameters, config):
    print(f"[Client {self.cid}] evaluate, config: {config}")
    self.net = set_parameters(self.net, parameters)
    loss, accuracy = test(self.net, self.valloader)
    print(f"[Client {self.cid}] loss:{loss}, Client {self.cid} accuracy:
↪{accuracy}")
    return float(loss), len(self.valloader), {"accuracy": float(accuracy)}

def client_fn(context: Context):
    """Returns a FlowerClient containing its data partition."""

    # Create the model
    net = generate_ann()
    #get the identification
    partition_id = int(context.node_config["partition-id"])
    #Take the appropriate part of the dataset
    trainloader = trainloaders[int(partition_id)]
    valloader = valloaders[int(partition_id)]
    #Create and return the Client
    return FlowerClient(partition_id, net, trainloader, valloader).to_client()

# Construct the ClientApp passing the client generation function
client_app = ClientApp(client_fn=client_fn)

from flwr.server import ServerApp, ServerAppComponents

from typing import Tuple, List
from flwr.common import Metrics

def weighted_average(metrics: List[Tuple[int, Metrics]]) -> Metrics:
    # Multiply accuracy of each client by number of examples used
    accuracies = [num_examples * m["accuracy"] for num_examples, m in metrics]
    examples = [num_examples for num_examples, _ in metrics]
    # Aggregate and return custom metric (weighted average)
    return {"accuracy": sum(accuracies) / sum(examples)}

def server_fn(context: Context):
    # instantiate the model

```

```

model = generate_ann()
params = get_parameters(model) # The federated model initial parameters
del model # this is not require but it saves a good amount of memory
# Convert model parameters to flwr.common.Parameters
global_model_init = flwr.common.ndarrays_to_parameters(params)

# Create FedAvg strategy
strategy = flwr.server.strategy.FedAvg(
    fraction_fit=1.0, # Sample 100% of available clients for training
    fraction_evaluate=0.5, # Sample 50% of available clients for
    ↪evaluation
    min_fit_clients=NUM_CLIENTS, # Never sample less than NUM_CLIENTS
    ↪clients for training
    min_evaluate_clients=NUM_CLIENTS//2, # Never sample less than
    ↪NUM_CLIENTS//2 clients for evaluation
    min_available_clients=NUM_CLIENTS, # Wait until all NUM_CLIENTS
    ↪clients are available
    initial_parameters=flwr.common.ndarrays_to_parameters(params), #
    ↪Initial parameters
    evaluate_metrics_aggregation_fn=weighted_average, # put the metric
    ↪aggregation for the evaluation
)

# Define ServerConfig
config=flwr.server.ServerConfig(num_rounds=10)

# Return the configuration and strategy for this server
return ServerAppComponents(strategy=strategy, config=config)

# Create Server
server_app = ServerApp(server_fn=server_fn)

flwr.simulation.run_simulation(
    server_app=server_app, client_app=client_app, num_supernodes=NUM_CLIENTS
)

```

/Users/albertolandi/anaconda3/envs/ml2/lib/python3.12/site-packages/keras/src/utils/file_utils.py:115: DeprecationWarning: Python 3.14 will, by default, filter extracted tar archives and reject files or modify their metadata. Use the filter argument to control this behavior.

```

archive.extractall(
INFO : Starting Flower ServerApp, config: num_rounds=10, no
round_timeout
INFO :
INFO : [INIT]
INFO : Using initial global parameters provided by strategy

```

```

INFO :      Starting evaluation of initial global parameters
INFO :      Evaluation returned no results (`None`)
INFO :
INFO :      [ROUND 1]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)

(ClientAppActor pid=65412) [Client 0] fit, config: {}
(ClientAppActor pid=65412) Epoch 1/5
1/3          3s 2s/step -
accuracy: 0.1875 - loss: 3.4580
2/3          0s 196ms/step -
accuracy: 0.0469 - loss: 4.2635
3/3          2s 166ms/step -
accuracy: 0.1393 - loss: 3.5347
2/3          0s 168ms/step -
accuracy: 0.1641 - loss: 3.0873
2/3          0s 94ms/step -
accuracy: 0.1250 - loss: 3.1855
3/3          0s 126ms/step -
accuracy: 0.1146 - loss: 3.3096
(ClientAppActor pid=65410) [Client 4] fit, config: {} [repeated 4x
across cluster]
(ClientAppActor pid=65410) Epoch 1/5 [repeated 20x across
cluster]

INFO :      aggregate_fit: received 5 results and 0 failures
WARNING :    No fit_metrics_aggregation_fn provided
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)

(ClientAppActor pid=65412) [Client 3] evaluate, config: {}
1/3          0s 42ms/step -
accuracy: 0.2188 - loss: 3.5549 [repeated 24x across cluster]

INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 2]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)

3/3          0s 41ms/step -
accuracy: 0.1007 - loss: 3.5723 [repeated 4x across cluster]
7/7          0s 16ms/step -
accuracy: 0.0930 - loss: 2.3369
(ClientAppActor pid=65412) [Client 3] loss:2.3192365169525146, Client 3
accuracy:0.10000000149011612
3/3          0s 138ms/step -
accuracy: 0.1784 - loss: 3.3691 [repeated 17x across cluster]
3/3          1s 41ms/step -
accuracy: 0.0964 - loss: 3.6094
2/3          0s 185ms/step -

```

```

accuracy: 0.0547 - loss: 4.4072 [repeated 16x across cluster]
3/3          0s 42ms/step -
accuracy: 0.1862 - loss: 3.2014 [repeated 4x across cluster]
2/3          0s 71ms/step -
accuracy: 0.0859 - loss: 3.6967
(ClientAppActor pid=65410) [Client 1] fit, config: {} [repeated 4x
across cluster]
(ClientAppActor pid=65412) Epoch 2/5 [repeated 11x across
cluster]
3/3          0s 160ms/step -
accuracy: 0.1328 - loss: 3.1040
(ClientAppActor pid=65410) [Client 2] evaluate, config: {}
1/3          0s 112ms/step -
accuracy: 0.1562 - loss: 2.9764 [repeated 17x across cluster]
2/3          0s 72ms/step -
accuracy: 0.1406 - loss: 3.0021 [repeated 3x across cluster]
7/7          0s 16ms/step -
accuracy: 0.1182 - loss: 2.3087
(ClientAppActor pid=65410) [Client 2] loss:2.3234922885894775, Client 2
accuracy:0.10499999672174454
3/3          0s 66ms/step -
accuracy: 0.2135 - loss: 3.1656
3/3          0s 155ms/step -
accuracy: 0.2135 - loss: 2.9585 [repeated 10x across cluster]
2/3          0s 207ms/step -
accuracy: 0.2500 - loss: 2.8419 [repeated 13x across cluster]
3/3          0s 50ms/step -
accuracy: 0.1172 - loss: 3.1468 [repeated 8x across cluster]
3/3          2s 43ms/step -
accuracy: 0.1875 - loss: 3.1888
3/3          0s 41ms/step -
accuracy: 0.1133 - loss: 3.3060

INFO :      aggregate_fit: received 5 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)
(ClientAppActor pid=65413) [Client 0] evaluate, config: {}

INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 3]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)

7/7          0s 13ms/step -
accuracy: 0.0659 - loss: 2.3471
(ClientAppActor pid=65413) [Client 0] loss:2.3359527587890625, Client 0
accuracy:0.07500000298023224
(ClientAppActor pid=65413) [Client 4] fit, config: {}

```

```

(ClientAppActor pid=65413) Epoch 5/5 [repeated 18x across
cluster]
7/7          0s 11ms/step -
accuracy: 0.0963 - loss: 2.3721
(ClientAppActor pid=65410) [Client 0] fit, config: {}
1/7          1s 195ms/step -
accuracy: 0.0625 - loss: 2.4558 [repeated 9x across cluster]
2/3          0s 114ms/step -
accuracy: 0.1562 - loss: 2.9919
3/3          2s 108ms/step -
accuracy: 0.1875 - loss: 2.9453
3/3          0s 42ms/step -
accuracy: 0.1497 - loss: 3.1886 [repeated 3x across cluster]
3/3          2s 139ms/step -
accuracy: 0.1901 - loss: 3.0379
2/3          0s 145ms/step -
accuracy: 0.1484 - loss: 2.8615
2/3          0s 176ms/step -
accuracy: 0.2734 - loss: 2.5204
(ClientAppActor pid=65410) [Client 3] evaluate, config: {}
3/3          0s 135ms/step -
accuracy: 0.1536 - loss: 2.8507
(ClientAppActor pid=65410) [Client 3] loss:2.344329595565796, Client 3
accuracy:0.09000000357627869
(ClientAppActor pid=65411) Epoch 5/5 [repeated 19x across
cluster]
3/3          1s 40ms/step -
accuracy: 0.1693 - loss: 3.1193
(ClientAppActor pid=65413) [Client 4] fit, config: {} [repeated 4x
across cluster]
1/3          0s 95ms/step -
accuracy: 0.3125 - loss: 2.5586 [repeated 20x across cluster]

INFO :      aggregate_fit: received 5 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)
(ClientAppActor pid=65412) [Client 4] evaluate, config: {}

INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 4]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)

7/7          0s 17ms/step -
accuracy: 0.1162 - loss: 2.3146
(ClientAppActor pid=65412) [Client 4] loss:2.3245849609375, Client 4
accuracy:0.10999999940395355
2/3          0s 101ms/step -

```

```

accuracy: 0.2500 - loss: 2.6459 [repeated 3x across cluster]
7/7          0s 14ms/step -
accuracy: 0.0995 - loss: 2.3445
3/3          0s 39ms/step -
accuracy: 0.3099 - loss: 2.6232 [repeated 5x across cluster]
3/3          1s 159ms/step -
accuracy: 0.1862 - loss: 2.8869 [repeated 16x across cluster]
2/3          0s 215ms/step -
accuracy: 0.1953 - loss: 2.8282 [repeated 12x across cluster]
(ClientAppActor pid=65410) Epoch 1/5 [repeated 10x across
cluster]
(ClientAppActor pid=65410) [Client 2] fit, config: {} [repeated 4x
across cluster]
1/3          0s 159ms/step -
accuracy: 0.1562 - loss: 3.3957 [repeated 10x across cluster]
2/3          0s 88ms/step -
accuracy: 0.1172 - loss: 3.2002
2/3          0s 203ms/step -
accuracy: 0.1562 - loss: 2.9667
(ClientAppActor pid=65413) [Client 1] evaluate, config: {}
(ClientAppActor pid=65413) [Client 1] loss:2.343479871749878, Client 1
accuracy:0.09000000357627869
2/3          0s 181ms/step -
accuracy: 0.1953 - loss: 3.2423 [repeated 4x across cluster]
3/3          0s 95ms/step -
accuracy: 0.1016 - loss: 3.0185
3/3          0s 155ms/step -
accuracy: 0.1367 - loss: 3.1343 [repeated 2x across cluster]
3/3          0s 121ms/step -
accuracy: 0.1315 - loss: 3.1994 [repeated 17x across cluster]
2/3          0s 148ms/step -
accuracy: 0.1172 - loss: 3.1930 [repeated 14x across cluster]
3/3          1s 43ms/step -
accuracy: 0.1979 - loss: 3.0783
3/3          0s 46ms/step -
accuracy: 0.2357 - loss: 2.8964

INFO :      aggregate_fit: received 5 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)

(ClientAppActor pid=65412) [Client 3] evaluate, config: {}
(ClientAppActor pid=65412) Epoch 5/5 [repeated 21x across
cluster]

INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 5]

```

```

INFO :      configure_fit: strategy sampled 5 clients (out of 5)

(ClientAppActor pid=65412) [Client 3] loss:2.430224657058716, Client 3
accuracy:0.09000000357627869
7/7          0s 9ms/step -
accuracy: 0.1162 - loss: 2.3538
(ClientAppActor pid=65412) [Client 1] fit, config: {} [repeated 2x
across cluster]
1/7          1s 175ms/step -
accuracy: 0.0625 - loss: 2.6197 [repeated 19x across cluster]
7/7          0s 12ms/step -
accuracy: 0.0963 - loss: 2.4730
3/3          2s 132ms/step -
accuracy: 0.1719 - loss: 2.8205
2/3          0s 162ms/step -
accuracy: 0.1719 - loss: 2.9526
3/3          0s 42ms/step -
accuracy: 0.2018 - loss: 2.9690 [repeated 3x across cluster]
3/3          2s 163ms/step -
accuracy: 0.2695 - loss: 2.7768 [repeated 2x across cluster]
2/3          0s 204ms/step -
accuracy: 0.1562 - loss: 2.9328
3/3          0s 127ms/step -
accuracy: 0.1966 - loss: 2.5880
2/3          0s 88ms/step -
accuracy: 0.3359 - loss: 2.3787
(ClientAppActor pid=65413) [Client 4] evaluate, config: {}
(ClientAppActor pid=65412) Epoch 5/5 [repeated 16x across
cluster]
(ClientAppActor pid=65413) [Client 4] loss:2.364698886871338, Client 4
accuracy:0.10999999940395355
2/3          0s 56ms/step -
accuracy: 0.2266 - loss: 2.8035
(ClientAppActor pid=65412) [Client 4] fit, config: {} [repeated 4x
across cluster]
1/3          3s 2s/step -
accuracy: 0.0938 - loss: 2.9490 [repeated 19x across cluster]
3/3          2s 42ms/step -
accuracy: 0.1328 - loss: 2.8749

INFO :      aggregate_fit: received 5 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)

3/3          0s 40ms/step -
accuracy: 0.2227 - loss: 2.8600
3/3          0s 40ms/step -
accuracy: 0.2435 - loss: 2.5237

```

```

(ClientAppActor pid=65412) [Client 2] evaluate, config: {}
2/3          0s 81ms/step -
accuracy: 0.2344 - loss: 2.8860 [repeated 2x across cluster]
3/3          0s 44ms/step -
accuracy: 0.2721 - loss: 2.3830 [repeated 5x across cluster]
3/3          0s 145ms/step -
accuracy: 0.1693 - loss: 2.8852 [repeated 13x across cluster]
2/3          0s 152ms/step -
accuracy: 0.1406 - loss: 2.8524 [repeated 12x across cluster]

INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 6]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)

7/7          0s 13ms/step -
accuracy: 0.1271 - loss: 2.4202
(ClientAppActor pid=65412) [Client 2] loss:2.475313186645508, Client 2
accuracy:0.115000000208616257
7/7          0s 10ms/step -
accuracy: 0.0963 - loss: 2.5263
(ClientAppActor pid=65412) Epoch 5/5 [repeated 9x across
cluster]
3/3          2s 94ms/step -
accuracy: 0.1914 - loss: 2.8745
(ClientAppActor pid=65413) [Client 2] fit, config: {} [repeated 4x
across cluster]
1/3          4s 2s/step -
accuracy: 0.1562 - loss: 2.9961 [repeated 7x across cluster]
2/3          0s 212ms/step -
accuracy: 0.3125 - loss: 2.4350
(ClientAppActor pid=65410) [Client 3] evaluate, config: {}
2/3          0s 100ms/step -
accuracy: 0.2422 - loss: 2.6710 [repeated 3x across cluster]
3/3          2s 115ms/step -
accuracy: 0.1654 - loss: 2.8954
3/3          3s 109ms/step -
accuracy: 0.2669 - loss: 2.5702 [repeated 7x across cluster]
2/3          0s 196ms/step -
accuracy: 0.2109 - loss: 2.5439 [repeated 8x across cluster]
3/3          1s 196ms/step -
accuracy: 0.2786 - loss: 2.7354
(ClientAppActor pid=65410) [Client 3] loss:2.478419542312622, Client 3
accuracy:0.090000000357627869
3/3          1s 96ms/step -
accuracy: 0.1602 - loss: 3.0024

```

```

(ClientAppActor pid=65413) Epoch 4/5 [repeated 19x across
cluster]
2/3          0s 52ms/step -
accuracy: 0.1172 - loss: 3.1034
3/3          1s 43ms/step -
accuracy: 0.2031 - loss: 2.7325
3/3          0s 41ms/step -
accuracy: 0.2096 - loss: 2.9181

INFO :      aggregate_fit: received 5 results and 0 failures

(ClientAppActor pid=65410) [Client 4] fit, config: {}
1/3          0s 102ms/step -
accuracy: 0.1562 - loss: 2.7812 [repeated 18x across cluster]

INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)

(ClientAppActor pid=65413) [Client 4] evaluate, config: {}
3/3          1s 398ms/step -
accuracy: 0.2448 - loss: 2.7988 [repeated 6x across cluster]
2/3          0s 73ms/step -
accuracy: 0.1719 - loss: 2.8483 [repeated 7x across cluster]

INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 7]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)

3/3          0s 96ms/step -
accuracy: 0.1849 - loss: 2.8942 [repeated 5x across cluster]
7/7          0s 18ms/step -
accuracy: 0.1162 - loss: 2.4318
(ClientAppActor pid=65413) [Client 4] loss:2.4429073333740234, Client 4
accuracy:0.10999999940395355
7/7          0s 19ms/step -
accuracy: 0.0590 - loss: 2.5800
(ClientAppActor pid=65412) [Client 3] fit, config: {}
(ClientAppActor pid=65412) Epoch 1/5 [repeated 7x across
cluster]
2/3          0s 69ms/step -
accuracy: 0.1875 - loss: 2.7206
2/3          0s 105ms/step -
accuracy: 0.2656 - loss: 2.7856
3/3          2s 98ms/step -
accuracy: 0.1549 - loss: 2.6018
2/3          0s 102ms/step -
accuracy: 0.1641 - loss: 2.5514
1/3          0s 82ms/step -
accuracy: 0.2500 - loss: 2.6278 [repeated 6x across cluster]

```

```

(ClientAppActor pid=65410) [Client 0] evaluate, config: {}
3/3          0s 159ms/step -
accuracy: 0.2513 - loss: 2.6652 [repeated 4x across cluster]
2/3          0s 152ms/step -
accuracy: 0.1797 - loss: 2.7422 [repeated 3x across cluster]
3/3          0s 145ms/step -
accuracy: 0.1745 - loss: 2.7580
3/3          3s 96ms/step -
accuracy: 0.3359 - loss: 2.5165 [repeated 4x across cluster]
3/3          3s 98ms/step -
accuracy: 0.2422 - loss: 2.7902 [repeated 2x across cluster]
(ClientAppActor pid=65410) [Client 0] loss:2.5492637157440186, Client 0
accuracy:0.070000000029802322
3/3          0s 68ms/step -
accuracy: 0.2240 - loss: 2.7356
3/3          1s 408ms/step -
accuracy: 0.3451 - loss: 2.3348
(ClientAppActor pid=65411) [Client 2] fit, config: {} [repeated 3x
across cluster]
(ClientAppActor pid=65411) Epoch 5/5 [repeated 19x across
cluster]
3/3          2s 43ms/step -
accuracy: 0.3424 - loss: 2.5004
2/3          0s 68ms/step -
accuracy: 0.3594 - loss: 2.4571 [repeated 3x across cluster]
INFO :      aggregate_fit: received 5 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)
2/3          0s 66ms/step -
accuracy: 0.2734 - loss: 2.7578 [repeated 6x across cluster]
1/3          0s 43ms/step -
accuracy: 0.1562 - loss: 2.7133 [repeated 21x across cluster]
(ClientAppActor pid=65412) [Client 0] evaluate, config: {}
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 8]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)
5/7          0s 15ms/step -
accuracy: 0.0518 - loss: 2.6611
(ClientAppActor pid=65412) [Client 0] loss:2.6038098335266113, Client 0
accuracy:0.070000000029802322
7/7          0s 10ms/step -
accuracy: 0.1162 - loss: 2.4739
3/3          1s 167ms/step -
accuracy: 0.1562 - loss: 2.8280 [repeated 2x across cluster]

```

```

2/3          0s 168ms/step -
accuracy: 0.1562 - loss: 2.8042 [repeated 2x across cluster]
3/3          0s 42ms/step -
accuracy: 0.2227 - loss: 2.6338 [repeated 8x across cluster]
(ClientAppActor pid=65412) [Client 1] fit, config: {} [repeated 5x
across cluster]
(ClientAppActor pid=65413) Epoch 1/5 [repeated 9x across
cluster]
2/3          0s 143ms/step -
accuracy: 0.1719 - loss: 2.5173
2/3          0s 288ms/step -
accuracy: 0.2734 - loss: 2.4310
7/7          0s 15ms/step -
accuracy: 0.0590 - loss: 2.6396
3/3          0s 96ms/step -
accuracy: 0.2240 - loss: 2.4194
1/3          0s 76ms/step -
accuracy: 0.2812 - loss: 2.6236 [repeated 4x across cluster]
2/3          0s 117ms/step -
accuracy: 0.2656 - loss: 2.7224
(ClientAppActor pid=65411) [Client 4] evaluate, config: {}
2/3          0s 81ms/step -
accuracy: 0.2344 - loss: 2.7536
(ClientAppActor pid=65411) [Client 4] loss:2.4867143630981445, Client 4
accuracy:0.10999999940395355
3/3          0s 118ms/step -
accuracy: 0.2344 - loss: 2.8458 [repeated 10x across cluster]
2/3          0s 106ms/step -
accuracy: 0.2500 - loss: 2.9512 [repeated 4x across cluster]
3/3          0s 83ms/step -
accuracy: 0.2422 - loss: 2.6176 [repeated 5x across cluster]
3/3          2s 39ms/step -
accuracy: 0.3346 - loss: 2.2550
3/3          0s 41ms/step -
accuracy: 0.2161 - loss: 2.8405

INFO :      aggregate_fit: received 5 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)

(ClientAppActor pid=65411) [Client 4] fit, config: {}
(ClientAppActor pid=65411) Epoch 5/5 [repeated 21x across
cluster]
(ClientAppActor pid=65413) [Client 4] evaluate, config: {}
2/3          0s 118ms/step -
accuracy: 0.2188 - loss: 2.9102 [repeated 2x across cluster]
3/3          0s 95ms/step -
accuracy: 0.2435 - loss: 2.7771 [repeated 2x across cluster]

```

```

1/3          0s 52ms/step -
accuracy: 0.1250 - loss: 2.8568 [repeated 19x across cluster]
2/3          0s 78ms/step -
accuracy: 0.3359 - loss: 2.4060 [repeated 9x across cluster]

INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 9]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)

7/7          0s 12ms/step -
accuracy: 0.1271 - loss: 2.5511
(ClientAppActor pid=65411) [Client 2] loss:2.625462293624878, Client 2
accuracy:0.11500000208616257
(ClientAppActor pid=65410) [Client 3] fit, config: {}
3/3          0s 105ms/step -
accuracy: 0.3346 - loss: 2.1978
2/3          0s 120ms/step -
accuracy: 0.3672 - loss: 2.1239
3/3          0s 48ms/step -
accuracy: 0.2057 - loss: 2.7377 [repeated 4x across cluster]
3/3          2s 82ms/step -
accuracy: 0.2370 - loss: 2.6323
3/3          0s 74ms/step -
accuracy: 0.2005 - loss: 2.4800
(ClientAppActor pid=65410) Epoch 3/5 [repeated 6x across
cluster]
3/3          0s 70ms/step -
accuracy: 0.2435 - loss: 2.5465
2/3          0s 65ms/step -
accuracy: 0.2969 - loss: 2.6919
3/3          0s 188ms/step -
accuracy: 0.2135 - loss: 2.6738
2/3          0s 112ms/step -
accuracy: 0.3203 - loss: 2.3051
(ClientAppActor pid=65411) [Client 2] evaluate, config: {}
2/3          0s 107ms/step -
accuracy: 0.1641 - loss: 2.6776 [repeated 2x across cluster]
7/7          0s 10ms/step -
accuracy: 0.1162 - loss: 2.5347
1/3          5s 3s/step -
accuracy: 0.1250 - loss: 2.8149 [repeated 10x across cluster]
2/3          0s 154ms/step -
accuracy: 0.2188 - loss: 2.5751
(ClientAppActor pid=65413) [Client 4] loss:2.5474491119384766, Client 4
accuracy:0.10999999940395355
2/3          0s 186ms/step -
accuracy: 0.2734 - loss: 2.3555

```

```

2/3          0s 192ms/step -
accuracy: 0.2500 - loss: 2.3684
(ClientAppActor pid=65411) [Client 1] fit, config: {} [repeated 3x
across cluster]
1/3          1s 650ms/step -
accuracy: 0.1875 - loss: 2.7209
3/3          0s 92ms/step -
accuracy: 0.2214 - loss: 2.5899
3/3          1s 193ms/step -
accuracy: 0.2370 - loss: 2.2292 [repeated 3x across cluster]
3/3          2s 43ms/step -
accuracy: 0.2747 - loss: 2.6498
3/3          0s 41ms/step -
accuracy: 0.2682 - loss: 2.3873

INFO :      aggregate_fit: received 5 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)

(ClientAppActor pid=65410) Epoch 5/5 [repeated 19x across
cluster]
(ClientAppActor pid=65410) [Client 0] evaluate, config: {}
2/3          0s 51ms/step -
accuracy: 0.2344 - loss: 2.4858
3/3          0s 117ms/step -
accuracy: 0.1562 - loss: 3.0456 [repeated 9x across cluster]
2/3          0s 134ms/step -
accuracy: 0.1562 - loss: 3.1279 [repeated 10x across cluster]
1/3          0s 44ms/step -
accuracy: 0.3438 - loss: 2.1052 [repeated 12x across cluster]

INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 10]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)

4/7          0s 18ms/step -
accuracy: 0.1237 - loss: 2.5406
7/7          0s 18ms/step -
accuracy: 0.0590 - loss: 2.7518
(ClientAppActor pid=65410) [Client 0] loss:2.7049448490142822, Client 0
accuracy:0.070000000029802322
7/7          0s 14ms/step -
accuracy: 0.1162 - loss: 2.5556 [repeated 2x across cluster]
(ClientAppActor pid=65411) [Client 2] fit, config: {} [repeated 3x
across cluster]
3/3          0s 45ms/step -
accuracy: 0.3177 - loss: 2.2609 [repeated 4x across cluster]
2/3          0s 98ms/step -

```

```

accuracy: 0.3438 - loss: 2.5241
(ClientAppActor pid=65413) Epoch 1/5 [repeated 4x across
cluster]
(ClientAppActor pid=65412) [Client 4] evaluate, config: {}
3/3          0s 155ms/step -
accuracy: 0.2760 - loss: 2.2631 [repeated 7x across cluster]
2/3          0s 190ms/step -
accuracy: 0.3047 - loss: 2.3922 [repeated 4x across cluster]
1/3          6s 3s/step -
accuracy: 0.2188 - loss: 2.1943 [repeated 12x across cluster]
(ClientAppActor pid=65412) [Client 4] loss:2.566208839416504, Client 4
accuracy:0.10999999940395355
2/3          0s 102ms/step -
accuracy: 0.2812 - loss: 2.4070 [repeated 3x across cluster]
(ClientAppActor pid=65413) [Client 3] fit, config: {} [repeated 2x
across cluster]
2/3          0s 56ms/step -
accuracy: 0.2891 - loss: 2.6908
3/3          2s 47ms/step -
accuracy: 0.1680 - loss: 2.8922
3/3          0s 44ms/step -
accuracy: 0.3919 - loss: 2.2194 [repeated 3x across cluster]
2/3          0s 104ms/step -
accuracy: 0.2344 - loss: 2.2951 [repeated 3x across cluster]
3/3          0s 43ms/step -
accuracy: 0.3268 - loss: 2.4819
(ClientAppActor pid=65412) Epoch 4/5 [repeated 20x across
cluster]

INFO :      aggregate_fit: received 5 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)
(ClientAppActor pid=65412) [Client 0] evaluate, config: {}

INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [SUMMARY]
INFO :      Run finished 10 round(s) in 97.59s
INFO :      History (loss, distributed):
INFO :          round 1: 2.321364402770996
INFO :          round 2: 2.340141177177429
INFO :          round 3: 2.334032416343689
INFO :          round 4: 2.397461771965027
INFO :          round 5: 2.476866364479065
INFO :          round 6: 2.496085524559021
INFO :          round 7: 2.545262098312378
INFO :          round 8: 2.5864557027816772

```

```

INFO :                round 9: 2.635576844215393
INFO :                round 10: 2.7039746046066284
INFO :      History (metrics, distributed, evaluate):
INFO :      {'accuracy': [(1, 0.10249999910593033),
INFO :                    (2, 0.08250000327825546),
INFO :                    (3, 0.10000000149011612),
INFO :                    (4, 0.10000000149011612),
INFO :                    (5, 0.10250000283122063),
INFO :                    (6, 0.08999999985098839),
INFO :                    (7, 0.08999999985098839),
INFO :                    (8, 0.11250000074505806),
INFO :                    (9, 0.08999999985098839),
INFO :                    (10, 0.09750000014901161)]}
INFO :

3/3          1s 404ms/step -
accuracy: 0.2734 - loss: 2.5314 [repeated 10x across cluster]
2/3          0s 721ms/step -
accuracy: 0.2344 - loss: 2.5939 [repeated 7x across cluster]
1/3          0s 49ms/step -
accuracy: 0.1250 - loss: 2.9336 [repeated 11x across cluster]
(ClientAppActor pid=65412) [Client 0] loss:2.732985496520996, Client 0
accuracy:0.07500000298023224
7/7          0s 15ms/step -
accuracy: 0.1340 - loss: 2.5925
3/3          0s 53ms/step -
accuracy: 0.2747 - loss: 2.6424
(ClientAppActor pid=65412) [Client 4] fit, config: {}
4/7          0s 17ms/step -
accuracy: 0.0495 - loss: 2.8346 [repeated 2x across cluster]
7/7          0s 17ms/step -
accuracy: 0.0626 - loss: 2.7824 [repeated 3x across cluster]
(ClientAppActor pid=65412) Epoch 5/5
(ClientAppActor pid=65413) [Client 2] evaluate, config: {}
1/7          1s 221ms/step -
accuracy: 0.1562 - loss: 2.5416
(ClientAppActor pid=65413) [Client 2] loss:2.6749637126922607, Client 2
accuracy:0.11999999731779099

```

4 Aggregation

To conclude this lesson, let's take a closer look at the key point of these strategies, which is the aggregation algorithm. These algorithms are responsible for combining the updates from the clients to generate the global model, and they are defined in the strategies as we have seen. Generally speaking, there are several types of aggregation that can be used in federated learning (Reddi et. al, 2020).

Here are the different types of aggregation that can be used in federated learning:

- Federated averaging (`flwr.server.strategy.FedAvg`): In this approach, each device computes an update to the model parameters based on its local data, and these updates are then averaged together to create the global model. This approach is simple and effective, but it can be sensitive to the size of the updates and the quality of the data on each device.
- Federated weighted averaging: This approach is similar to federated averaging, but each device’s update is given a different weight based on the size of its data set or the quality of its data. This can help to give more influence to devices with larger or higher-quality data.
- Federated averaging with momentum (`flwr.server.strategy.FedAvgM`): This approach is similar to federated averaging, but it incorporates a momentum term in order to smooth out the updates and help the model converge more quickly.
- Federated stochastic gradient descent(`flwr.server.strategy.FedAdagrad`): In this approach, each device computes an update to the model parameters based on a small batch of its local data, rather than the entire data set. This can help to reduce the communication overhead and improve the convergence rate of the model.
- Federated ADAM (`flwr.server.strategy.FedAdam`): This approach is a variant of federated stochastic gradient descent that uses the ADAM optimization algorithm to adaptively adjust the learning rate based on the gradient and second moment estimates.

All of the previously mentioned aggregation methods, except for Federated Weighted Averaging, are implemented in the `flwr` framework and can be used with the different strategies. Additionally, there are other less common aggregation methods that can be employed. The choice of aggregation method will ultimately depend on the specific characteristics of the data and the requirements of the task at hand.

References

- Hard, A., Konečný, J., McMahan, H. B., Richemond-Barakat, C., Sivek, J. S., & Talwar, K. (2018). Federated learning: Strategies for improving communication efficiency. arXiv preprint arXiv:1812.02903.
- Li, Y., Bonawitz, K., & Talwar, K. (2020). Fedprox: An optimizer for communication-efficient federated learning. arXiv preprint arXiv:2002.04283.
- McMahan, H. B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B. A. (2016). Communication-efficient learning of deep networks from decentralized data. arXiv preprint arXiv:1602.05629.
- Yoon, J., Hard, A., Konečný, J., McMahan, H. B., & Sohl-Dickstein, J. (2018). Federal regression: A simple and scalable method for heterogeneous federated learning. arXiv preprint arXiv:1812.03862.
- Reddi, S., Charles, Z., Zaheer, M., Garrett, Z., Rush, K., Konečný, J., Kumar, S. and McMahan, H.B., 2020. Adaptive federated optimization. arXiv preprint arXiv:2003.00295.